

FATEC SÃO CAETANO DO SUL ANTONIO RUSSO

Deysi Ticona RA: 1680972521042

Laura Alves Silva RA: 1680972521041

Lucas Negrelli RA: 1680972521018

Luiz Henrique RA: 1680972521001

Obed Sarmiento RA: 1680972521034

Renato Miranda RA: 1680972521023

IN-SECURITY V1.0

São Caetano do Sul – SP

Junho/2026

1. Introdução

O projeto IN-SECURITY — Secure Inventory System v1.0 é um sistema de gerenciamento de inventário desenvolvido na linguagem Python, com o objetivo de aplicar conceitos de programação modular, estrutura de dados, manipulação de arquivos, algoritmos de busca e ordenação, além de mecanismos básicos de segurança da informação.

O sistema permite que usuários autenticados realizem operações de cadastro, atualização, remoção e consulta de produtos, armazenando as informações em um arquivo CSV protegido por mecanismos de cifragem.

2. Hashing de credenciais e controle de acesso (sha-256)

Mecânica de Baixo Nível: O sistema anula o armazenamento de credenciais em texto puro. Utiliza a função de hash criptográfico unidirecional SHA-256 via biblioteca **hashlib**. Durante a inicialização ou alteração de credenciais, o nome de usuário e a senha recebem *encoding* UTF-8, são processados pelo algoritmo e convertidos em representações hexadecimais de 64 caracteres. O sistema grava o estado no arquivo **login.txt** utilizando o formato estrito

HASH_USER|HASH_SENHA. A autenticação ocorre pela comparação booleana das saídas do hash das entradas do terminal em tempo real contra o armazenamento persistido.

Gargalos e Vetores de Ataque: A implementação falha ao não gerar nem anexar um *salt* criptográfico aleatório aos *inputs* antes da execução do hashing. Este axioma expõe o banco de credenciais a ataques de dicionário pré-computados e *Rainbow Tables*. Um invasor que obtenha acesso de leitura ao **login.txt** reverte hashes comuns instantaneamente. Adicionalmente, a verificação de equivalência utiliza operadores lógicos padrão (**==**), o que não previne vazamento de tempo computacional (*timing attacks*), permitindo inferência do hash byte a byte.

3. Criptografia de dados em repouso (simétrica)

Mecânica de Baixo Nível: O mascaramento dos dados do inventário no arquivo CSV opera através de uma Cifra de César, uma cifragem de deslocamento por substituição monoalfabética. O algoritmo injeta **k=7** como chave estática. O

processamento ocorre via aritmética modular (módulo 26) aplicada estritamente sobre os intervalos numéricos da tabela ASCII correspondentes aos caracteres alfabéticos. A fórmula base da translação é $(\text{ord}(\text{char}) - \text{ord}('a') + \text{chave}) \% 26 + \text{ord}('a')$. A descryptografia exige apenas a rotação inversa $(26 - k)$.

Gargalos e Vetores de Ataque: O espaço de chaves contínuo é estático e limitado a 25 permutações, permitindo a quebra do algoritmo por força bruta de forma trivial. A vulnerabilidade crítica reside no fato de o código ignorar ativamente caracteres não-alfabéticos. A consequência física no disco é que volumes (inteiros), precificações numéricas (**float**) e identificadores de produto vazam integralmente em texto puro dentro do **inventario.csv**. A preservação da estrutura dos metadados expõe o banco a ataques de análise de frequência e extração direta de inteligência financeira sem necessidade de decifrar o texto.

4. Validação de entradas e prevenção de estado inválido

Mecânica de Baixo Nível: A integridade da tabela de alocação em RAM (dicionário de inventário) é imposta no módulo **inventory.py** por meio de coerção estrita de tipos (*Type Casting*) na borda de *input*.

- **Tipagem Forte:** Entradas brutas (**str**) para quantidades, preços e identificadores são submetidas a conversões compulsórias (**int()**, **float()**). Qualquer desvio da assinatura numérica dispara uma interrupção arquitetural (**ValueError**) que é capturada pelo bloco **try/except**, abortando o fluxo para prevenir injeções de estado malformado.
- **Controle de Fronteiras:** Limites de magnitude estáticos são validados antes da gravação na estrutura de dados (**ex: qtd < 0, preco < 0**), garantindo a inexistência de estados quantitativos negativos que corromperiam cálculos paralelos.
- **Sanitização Base:** Caracteres de escape e *whitespaces* periféricos sofrem remoção forçada usando **.strip()**, e o estado booleano opera sob validação binária *upper-case* rigorosa ("S" / "N").

Gargalos e Vetores de Ataque: Ausência de *Clamping* (limites superiores restritivos). A validação bloqueia injeções alfanuméricas e números negativos, mas aceita magnitudes astronômicas (ex: injeção de ponto flutuante **9e99**). Essa falha

viabiliza ataques de Negação de Serviço Lógica (DoS): o processamento linear do relatório estatístico executará multiplicações massivas (**qtd * preco**), causando estouro de acumuladores em memória (*Integer/Float Overflow*) ou paralisia da CPU.

5. Mitigação de força bruta e telemetria

Mecânica de Baixo Nível: As tentativas de comprometimento interativo sofrem bloqueio na origem da *shell*.

- **Lockout de Sessão:** O fluxo possui um contador atômico que restringe o operador a 3 ciclos de tentativa (**MAX_TENTATIVAS**). O esgotamento força um evento **sys.exit()**, derrubando o processo e neutralizando ferramentas de *brute-force* acopladas via *stdin*.
- **Isolamento Visual:** Senhas capturadas invocam o módulo **getpass**, anulando o *echo* do TTY no *stdout* para inviabilizar técnicas passivas de *shoulder surfing*.
- **Imutabilidade de Log Operacional:** Todo evento de acesso despacha um sinal ISO-8601 submetido a um descritor de arquivo invariavelmente em modo *append* ("a").

Gargalos e Vetores de Ataque: O **security.log** reside no sistema de arquivos local em texto puro, desprovido de chaves de assinatura criptográfica ou *syslog* remoto. A trava de limite de tentativas opera exclusivamente na memória volátil do processo; reiniciar o *script* zera o contador. Além disso, um ator malicioso com acesso ao *shell* possui o mesmo nível de privilégio da aplicação, podendo executar *wipes* binários no registro de auditoria, obliterando a telemetria sem romper a integridade sistêmica geral.

6. Persistência volátil e gerenciamento de estado (lazy save)

Mecânica de Baixo Nível: A arquitetura de dados opera em um modelo de estado desconectado (*Lazy Save*) para minimizar latência de I/O. Durante a execução do sistema, as rotinas CRUD interagem exclusivamente com um dicionário Python alocado em memória RAM (camada de texto puro). A sincronização de blocos para o disco rígido, que engatilha o processo recursivo de cifragem para gerar o arquivo **inventario.csv**, é retida em espera. A persistência só é comitada quando o usuário

aciona a terminação intencional do sistema (Opção 0), condicionado à flag booleana de estado **modificado**.

Gargalos e Vetores de Ataque: A latência proposital entre a transação em RAM e a gravação em disco cria uma janela crítica de perda de dados. Falhas de energia, interrupções por sinais de hardware (SIGKILL) ou travamentos do núcleo (Kernel Panic) causam a volatilização instantânea de todas as operações não salvas. A aplicação também carece de mecanismos de *lock* (exclusão mútua); acessos concorrentes por múltiplas instâncias sobrescreverão o banco de dados desordenadamente, corrompendo a integridade do inventário.

7. Backdoors intencionais e destruição forense (protocolo zero)

Mecânica de Baixo Nível: A base de código contém uma rotina clandestina de autodestruição acoplada ao manipulador de *inputs* do menu principal (ID "99"). Após um desafio baseado em força-bruta invertida da Cifra de César, o sistema engatilha um processo de obliteração forense. O algoritmo de deleção mapeia o tamanho exato dos arquivos vitais (**login.txt**, **inventario.csv**, **security.log**), abre descritores binários sobressalentes (**open(..., "wb")**) e injeta bytes nulos (**\x00**) no setor físico do disco antes de desvincular o nóduo com **os.remove()**. Em seguida, executa um *flush* na RAM via **inventario.clear()**.

Gargalos e Vetores de Ataque: O módulo é um *backdoor* arquitetural materializado. Ele consolida o princípio de que a aplicação opera sob uma violação de privilégio mínimo: o processo detém autoridade em nível de sistema operacional (SO) para trancar e destruir os próprios arquivos de log e infraestrutura de segurança. Qualquer funcionário autenticado — ou atacante que alcance o menu — munido do conhecimento do atalho numérico, pode iniciar um ataque catastrófico de deleção irrecuperável que extingue o banco de dados financeiro e destrói as provas de auditoria simultaneamente.

8. Conclusão

Ao desenvolver o IN-SECURITY possibilitou a aplicação prática de diversos conceitos de programação e segurança da informação. A arquitetura modular tornou o código mais organizado e facilitou a separação dos componentes utilizados.

Apesar de utilizar mecanismos de proteção como SHA-256, controle de tentativas de login e cifragem de dados, a análise técnica identificou limitações importantes, como a ausência de salt no armazenamento de credenciais e a utilização de uma cifra de baixa complexidade para o inventário.